

LAB 1B: DATA MODELING & DESIGN

JSON/XML, MYSQL & MONGODB WITH MONGOOSE ODM

Course: Web Application and Database Development Practicum (Backend & Database)

1. GENERAL INFORMATION AND LAB REGULATIONS

a. Lab Objectives

1. Understand and confidently work with the most common data-interchange formats: **JSON** and **XML** in Node.js.
2. Compare the strengths and weaknesses of the **relational database model (SQL - MySQL)** versus the **document-oriented NoSQL model (MongoDB)**.
3. Connect to and query MySQL using the **mysql2** driver in Node.js (with **Connection Pooling** and **Prepared Statements**).
4. Master **Mongoose**, the Object Data Modeling (ODM) library for Node.js, to design Schemas, Models, data validation, middleware hooks, and population (relationships) for MongoDB.

b. Tools & Libraries Used

Node.js (LTS), VS Code, MySQL Server (Laragon), MongoDB Atlas / Mongo Compass, and the npm packages **mysql2**, **mongoose**, **xml2js**, and **dotenv**.

2. CORE THEORY SUMMARY

CORE CONCEPTS: JSON/XML, SQL VS NOSQL & MONGOOSE ODM

In backend web design, JSON (JavaScript Object Notation) is the most widely used data-interchange format thanks to its lightweight, human-readable structure and natural compatibility with JavaScript. XML (eXtensible Markup Language) is still used in legacy enterprise systems and in e-invoice / banking integrations. As for databases, SQL (MySQL) enforces strict data integrity under the ACID model (Atomicity, Consistency, Isolation, Durability) on top of a fixed schema, while NoSQL (MongoDB) supports a flexible, schema-less / dynamic schema that is optimized for high-throughput reads/writes and horizontal scaling. The Mongoose library acts as an ODM, providing a

layer for schema management, data constraints, and relationships between collections in MongoDB.

Table 2.1: Comparing SQL (MySQL) and NoSQL Document (MongoDB) Databases

Criteria	SQL Database (MySQL)	NoSQL Document (MongoDB)
Data structure	Tables, rows & columns	Collections & documents
Storage format	Fixed, two-dimensional relational model	Flexible BSON / JSON documents
Schema constraints	Fixed schema (strict schema via DDL)	Dynamic schema (flexible / schema-less)
Interaction in Node.js	SQL queries (mysql2) or an ORM (Sequelize/Prisma)	Direct driver or an ODM (mongoose)
Typical use cases	Financial transactions, inventory management, systems that require ACID	Content systems, e-commerce catalogs, Big Data, real-time apps

3. IN-CLASS EXERCISES

Exercise 1: Handling JSON and XML Data in Node.js

Objective: Become proficient in reading, writing, parsing, and stringifying JSON data, as well as converting between XML and JSON using the **xml2js** library.

Instructions:

1. Install the XML conversion library: **npm install xml2js**
2. Create a file named **json_xml_demo.js** and run the code below:

Step 1: Import modules

```
// json_xml_demo.js - Handling JSON and XML data in Node.js
const fs = require('fs');
const xml2js = require('xml2js');
```

Step 2: Create a JavaScript object

```
const productData = {  
  id: "PROD-1001",  
  name: "Sony WH-1000XM5 Bluetooth Headphones",  
  price: 8500000,  
  inStock: true,  
  categories: ["Audio", "Electronics", "Wireless"],  
  supplier: {  
    name: "Sony Vietnam",  
    contact: "support@sony.com.vn"  
  }  
};
```

Step 3: Convert the JS object to JSON format

```
const jsonString = JSON.stringify(productData, null, 2);  
console.log("=== 1. FORMATTED JSON STRING ===");  
console.log(jsonString);  
  
// Write the JSON to product.json  
fs.writeFileSync('product.json', jsonString, 'utf8');
```

Step 4: Convert the JS object to XML format

```
const builder = new xml2js.Builder({ rootName: 'Product' });  
const xmlData = builder.buildObject(productData);  
console.log("\n=== 2. CONVERTED TO XML FORMAT ===");  
console.log(xmlData);  
  
// Write the XML to product.xml  
fs.writeFileSync('product.xml', xmlData, 'utf8');
```

Step 5: Read the XML file and parse it back into a JS object (xml2js parser)

```
fs.readFile('product.xml', 'utf8', (err, data) => {
```

```
if (err) throw err;
xml2js.parseString(data, { explicitArray: false }, (err, result) => {
  if (err) throw err;
  console.log("\n=== 3. XML FILE PARSED BACK INTO A JS OBJECT ===");
  console.log(result.Product);
  console.log(`Parsed product name: ${result.Product.name}`);
});
});
```

After running the code above, explain the structural differences between the JSON and XML representations.

EXTENDED REQUIREMENT: Add a function **validateAndMerge(jsonPath)** that performs the following tasks:

1. Read the **product.json** file that was just written.
2. Check that the **price** field is valid (must be a positive number > 0). If valid, calculate a 10% discounted price and add it as a new **discountPrice** field.
3. Write the merged object out to **product_final.json**.

Exercise 2: Connecting to and Querying MySQL in Node.js (mysql2 Driver)

Objective: Use the **mysql2/promise** library to create a connection pool and safely execute DDL/DML statements using prepared statements (to prevent SQL injection).

Instructions:

1. Install the libraries: **npm install mysql2 dotenv**
2. Start MySQL Server and create the **store_db** database using a MySQL client.
3. Create a **.env** file to store the connection settings:

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD= // enter your DB password
DB_NAME=store_db
DB_PORT=3306
```

4. Create a **mysql_demo.js** file to establish the connection and perform CRUD operations:

Step 1: Import modules

```
require('dotenv').config();  
const mysql = require('mysql2/promise');
```

Step 2: Initialize the connection pool

```
const pool = mysql.createPool({  
  host: process.env.DB_HOST,  
  user: process.env.DB_USER,  
  password: process.env.DB_PASSWORD,  
  database: process.env.DB_NAME,  
  port: process.env.DB_PORT,  
  waitForConnections: true,  
  connectionLimit: 10,  
  queueLimit: 0  
});
```

Step 3: Perform basic operations

```
async function main() {  
  try {  
    console.log('Connecting to MySQL Database...');  
  
    // Create the 'categories' table if it doesn't exist  
    await pool.execute(`  
      CREATE TABLE IF NOT EXISTS categories (  
        id INT AUTO_INCREMENT PRIMARY KEY,  
        name VARCHAR(100) NOT NULL UNIQUE,  
        description TEXT,  
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
      )  
    `);  
    console.log("-> 'categories' table is ready.");  
  
    // Insert data using a prepared statement  
    const insertSql = `INSERT INTO categories (name, description) VALUES (?, ?)`;  
    const [insertResult] = await pool.execute(insertSql, ['Food', 'Daily essentials']);
```

```

console.log(`-> Inserted category ID: ${insertResult.insertId}`);

// Select data
const [rows] = await pool.execute('SELECT * FROM categories WHERE name = ?', ['Food']);
console.log(`-> Query result:`, rows);

// Update data
const [updateResult] = await pool.execute(
  'UPDATE categories SET description = ? WHERE id = ?',
  ['Food, beverages and fresh food', insertResult.insertId]
);
console.log(`-> Number of updated rows: ${updateResult.affectedRows}`);
} catch (error) {
  console.error('MySQL error:', error.message);
} finally {
  await pool.end(); // Close connection
}
}

main();

```

EXTENDED REQUIREMENT: Build an **items** table with a foreign key linking to **categories**, then complete the following questions:

- **Question 1 (Filter & sort):** Write a query that retrieves products priced $\geq 500,000$ VND AND with **quantity** > 0 , sorted by price in descending order (**ORDER BY price DESC**).
- **Question 2 (Wildcard / LIKE search):** Write a function that searches for products by a given keyword (using a prepared statement with the **LIKE '%keyword%'** operator). Retrieve the products whose name contains “Gaming” or “Wireless”.
- **Question 3 (Aggregate functions):** Calculate the **total stock quantity** (**SUM(quantity)**), **average price** (**AVG(price)**), and **total number of items** (**COUNT(*)**) across the entire **items** table.
- **Question 4 (Category statistics - GROUP BY & HAVING):** For each category, calculate the total number of items and the total inventory value (**SUM(price * quantity)**). Only show categories whose total inventory value exceeds **10,000,000 VND** (using **HAVING**).

- **Question 5 (Outer join - LEFT JOIN):** Write a **LEFT JOIN** between **categories** and **items** to find the categories that **do not yet have any products** (check for **items.id IS NULL**).
- **Question 6 (Update data - UPDATE & return the new values):** Write a function that increases an item's stock quantity (**quantity = quantity + ?**) and updates its price for a given **item_id**. After the **UPDATE**, run a **SELECT** statement to return the updated item.
- **Question 7 (Subquery):** Write a SQL statement that uses a subquery to find the most expensive item (**price = (SELECT MAX(price) FROM items)**).
- **Question 8 (Delete data & handle foreign key constraints - DELETE & exception handling):** Write a function that deletes a category by **category_id**. Use a try/catch block to catch the foreign key constraint error if the category still has child items, and print a friendly error message for the user.

For each question, please provide:

1. A short explanation (2-3 sentences) of the reasoning/steps behind the SQL statement and Node.js code.
2. A screenshot of the source code (in VS Code).
3. A screenshot of the server output in the terminal.
4. A screenshot of the corresponding query result taken directly from MySQL Workbench / Laragon / CLI, for cross-checking.

Sample data for the extended exercise

```
async function setupDatabaseAndSeedData(pool) {
  // Create the items table with a foreign key referencing categories
  await pool.query(`
    CREATE TABLE IF NOT EXISTS items (
      id INT AUTO_INCREMENT PRIMARY KEY,
      category_id INT NOT NULL,
      item_name VARCHAR(150) NOT NULL,
      price DECIMAL(12,2) NOT NULL,
      quantity INT DEFAULT 0,
      FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE
      RESTRICT
    );
  `);

  // Clear old data so the environment always resets to a known state
```

```

await pool.query('SET FOREIGN_KEY_CHECKS = 0;');
await pool.query('TRUNCATE TABLE items;');
await pool.query('TRUNCATE TABLE categories;');
await pool.query('SET FOREIGN_KEY_CHECKS = 1;');

// Seed the categories table
await pool.query(`
  INSERT INTO categories (name, description) VALUES
  ('Food', 'Daily essentials and groceries'),
  ('Electronics', 'Gadgets, phones and computers'),
  ('Clothing', 'Apparel and fashion items'),
  ('Books', 'Educational and entertainment books'),
  ('Home & Living', 'Furniture and home appliances'),
  ('Sports & Outdoors', 'Sporting goods and outdoor equipment');
`);

// Seed the items table
// Note: 'Sports & Outdoors' intentionally has no items, for the
// LEFT JOIN exercise (Question 5).
await pool.query(`
  INSERT INTO items (category_id, item_name, price, quantity) VALUES
  (1, 'Apple', 25000.00, 100),
  (1, 'Milk', 32000.00, 50),
  (1, 'Bread', 15000.00, 30),
  (2, 'Smartphone', 5500000.00, 15),
  (2, 'Wireless Mouse', 250000.00, 40),
  (3, 'T-Shirt', 120000.00, 60),
  (3, 'Jeans', 350000.00, 25),
  (4, 'Node.js Programming', 180000.00, 20),
  (5, 'Desk Lamp', 150000.00, 35),
  (5, 'Coffee Mug', 45000.00, 80);
`);

console.log('Table structure and sample data initialized successfully!');
}

```

Exercise 3: NoSQL Data Modeling with Mongoose ODM in Node.js

Objective: Build a Mongoose schema and model, configure data validation rules, write middleware hooks (pre-save) to transform data, and perform CRUD operations against MongoDB Atlas / a local instance.

Instructions:

1. Install the Mongoose library: **npm install mongoose**
2. Add the connection string to **.env**:
MONGO_URI=mongodb://localhost:27017/shop_mongoose_db
3. Create a **mongoose_demo.js** file and run the code below:

Step 1: Import modules

```
require('dotenv').config();  
const mongoose = require('mongoose');
```

Step 2: Connect to the MongoDB database

```
const MONGO_URI = process.env.MONGO_URI ||  
'mongodb://127.0.0.1:27017/shop_mongoose_db';  
  
mongoose.connect(MONGO_URI)  
  .then(() => console.log("-> Connected to MongoDB successfully via Mongoose  
ODM!"))  
  .catch(err => console.error("MongoDB connection error:", err));
```

Step 3: Define the Mongoose schema with validation rules

```
const userSchema = new mongoose.Schema({  
  fullName: {  
    type: String,  
    required: [true, 'Full name is required'],  
    trim: true,  
    minlength: [2, 'Full name must be at least 2 characters long']  
  },  
  email: {  
    type: String,  
    required: [true, 'Email is required'],  
    unique: true,
```

```

    lowercase: true,
    match: [/^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/, 'Invalid email format']
  },
  age: {
    type: Number,
    min: [18, 'User age must be at least 18'],
    max: [100, 'Invalid age']
  },
  role: {
    type: String,
    enum: ['user', 'admin', 'manager'],
    default: 'user'
  },
  isActive: {
    type: Boolean,
    default: true
  }
}, {
  timestamps: true // Automatically add createdAt and updatedAt fields
});

```

Step 4: Add a pre-save middleware hook

```

userSchema.pre('save', function() {
  console.log(`[Middleware Pre-save] Preparing to save user: ${this.fullName}`);
});

```

Step 5: Create the model from the schema

```

const User = mongoose.model('User', userSchema);

```

Step 6: Run CRUD operations

```

async function runMongooseCRUD() {
  try {
    // Clear old data
    await User.deleteMany({});
  }
}

```

```

// --- C - CREATE ---
const newUser = await User.create({
  fullName: "Nguyen Van Hung",
  email: "hung.nguyen@example.com",
  age: 22,
  role: "admin"
});
console.log("1. [CREATE] Successfully created new user:", newUser);

// --- R - READ ---
const foundUser = await User.findOne({ email: "hung.nguyen@example.com" });
console.log("2. [READ] Found user by email:", foundUser.fullName);

// --- U - UPDATE ---
const updatedUser = await User.findByIdAndUpdate(
  newUser._id,
  { age: 23, role: "manager" },
  { returnDocument: 'after', runValidators: true } // Return the new document & enable
validation
);
console.log("3. [UPDATE] Successfully updated user:", updatedUser);

// --- D - DELETE ---
// await User.findByIdAndDelete(newUser._id);
// console.log("4. [DELETE] User deleted successfully.");
} catch (error) {
  console.error("Mongoose validation/operation error:", error.message);
} finally {
  await mongoose.connection.close();
  console.log("-> Mongoose connection closed.");
}
}

runMongooseCRUD();

```

EXTENDED REQUIREMENT: Advanced data modeling and querying with Mongoose ODM

- **Question 1 (Custom validation with Regex):** Add a **phone** field to **userSchema** with a custom validator that uses a regular expression to check for a valid Vietnamese phone number (10 digits, starting with 03, 05, 07, 08, or 09). Try inserting a user with a badly formatted phone number to trigger the validation error.
- **Question 2 (Virtual properties & formatting):** Create a virtual field **displayInfo** that automatically combines **fullName**, **email**, and **role** (e.g., “Nguyen Van Hung <hung.nguyen@example.com> [ADMIN]”), and configure the schema to include this virtual field when serialized to JSON.
- **Question 3 (Static methods):** Define a static method **findActiveByRole(roleName)** on **userSchema** that finds all users with the given **role** who are also active (**isActive: true**), sorted alphabetically by name (A–Z).
- **Question 4 (Instance methods / soft delete):** Add an **isDeleted** field (default: false) and define an instance method **softDelete()** on **userSchema** that sets **isDeleted = true**, **isActive = false**, and saves the document back to the database.
- **Question 5 (Middleware hooks - pre-save & query hook):** Write a pre-save hook (**pre('save')**) that simulates hashing the password before it is saved to the database. Then write a pre-find hook (**pre(/^find/)**) that automatically filters out documents with **isDeleted: true** from every find query.
- **Question 6 (Embedded sub-documents):** Create a sub-schema **addressSchema** (street, city, isDefault) and add an array field **addresses: [addressSchema]** to **userSchema**. Write a function that adds a new address to the addresses array of a specific user.
- **Question 7 (References & population):** Create a **postSchema** (title, content, author [ObjectId ref 'User']). Insert 2 sample posts and run a query using **.populate('author', 'fullName email role')** to retrieve the author's details.
- **Question 8 (Pagination & sorting):** Write a function **getPaginatedUsers(page, limit)** that accepts a page number and page size, and uses **.skip()**, **.limit()**, and **.sort({ createdAt: -1 })** to return the most recent users along with metadata (currentPage, totalPages, totalUsers).
- **Question 9 (Aggregation Framework):** Use **User.aggregate()** to calculate the number of users, average age (**avgAge**), maximum age (**maxAge**), and minimum age (**minAge**), grouped by **role**.
- **Question 10 (Security & query optimization - selection & lean queries):** Run a query to list users using **.select('-password')** to hide the sensitive password field, combined with **.lean()** to convert the result from Mongoose documents into plain JS objects for better memory efficiency.

4. HOMEWORK EXERCISES

Homework 1: Managing Order Transactions with SQL Transaction Integrity

Database schema & requirements: Create a database named `store_transaction_db` with 4 closely related tables:

- (1) `customers` (`id` INT PK, `full_name` VARCHAR(100), `balance` DECIMAL(12,2));
- (2) `products` (`id` INT PK, `product_name` VARCHAR(150), `price` DECIMAL(12,2), `stock` INT);
- (3) `orders` (`id` INT PK, `customer_id` INT FK, `total_amount` DECIMAL(12,2), `created_at` DATETIME);
- (4) `order_items` (`id` INT PK, `order_id` INT FK, `product_id` INT FK, `quantity` INT, `price` DECIMAL(12,2)).

Order transaction workflow: Write a Node.js file `homework_sql_transaction.js` that opens a SQL transaction (**beginTransaction**) and executes 5 strict steps:

1. **Step 1:** Check that the customer's balance $\geq$ the order total AND the product's stock $\geq$ the quantity being purchased.
2. **Step 2:** Deduct the amount from the customer's balance (**UPDATE customers SET balance = balance - total**).
3. **Step 3:** Deduct the quantity from the product's stock (**UPDATE products SET stock = stock - quantity**).
4. **Step 4:** Insert a new order record into **orders**.
5. **Step 5:** Insert the corresponding line item(s) into **order_items**.

*If all steps succeed: call **conn.commit()**. If any error occurs (insufficient balance / out of stock): call **conn.rollback()** to safely restore the data.*

Homework 2: Designing a Mongoose Schema for Course Registration (Student & Course)

Description: Write a Node.js file `homework_many_to_many.js` that defines 2 schemas with a Many-to-Many relationship: **Student** (with `courses`: [{ type: Schema.Types.ObjectId, ref: 'Course' }]) and **Course** (with `students`: [{ type: Schema.Types.ObjectId, ref: 'Student' }], `maxStudents`: Number, `availableSlots`: Number).

ADDITIONAL ADVANCED REQUIREMENTS (2 MANDATORY QUESTIONS):

1. **Enroll in a course - enrollCourse(studentId, courseId):**

- Check whether the student is already enrolled. Check that the course still has `availableSlots > 0`.
- If valid: add `courseId` to `Student.courses` AND `studentId` to `Course.students` at the same time, and decrease `availableSlots` by 1.

2. Drop a course - `dropCourse(studentId, courseId)`:

- Remove `courseId` from the `Student.courses` array and `studentId` from the `Course.students` array.
- Return 1 slot to the course (`availableSlots = availableSlots + 1`).

Homework 3: Reading Invoice Data from `invoices.json`, Validating It, and Syncing It into MongoDB via the Invoice Mongoose Model

Task: Write a file `homework_json_to_mongoose.js` that syncs invoice data into MongoDB:

1. Read the `invoices.json` file, which contains a list of invoices.
2. Define an **Invoice** Mongoose schema with validation rules (required fields, a valid email format, at least one item per invoice, and an allowed set of payment methods).
3. Validate and insert the entire parsed array into the **invoices** collection in MongoDB through the **Invoice** model.

Sample data file: `invoices.json`

```
[
  {
    "invoiceCode": "INV-2026-001",
    "customerName": "Nguyen Van Hung",
    "customerEmail": "hung.nguyen@example.com",
    "items": [
      { "productName": "Laptop Dell XPS 15", "quantity": 1, "price": 35000000 },
      { "productName": "Logitech MX Master 3S Mouse", "quantity": 1, "price": 2500000 }
    ]
  },
  {
    "invoiceCode": "INV-2026-002",
    "customerName": "Tran Thi Mai",
    "customerEmail": "mai.tran@example.com",
    "items": [
```

```
{ "productName": "Keychron K2 Mechanical Keyboard", "quantity": 2, "price":  
1800000 }  
],  
"paymentMethod": "Momo"  
}  
]
```